Object Oriented Programming (OOP) *Represent the real world*

OOP is a method of programming based on a hierarchy of classes, and well-defined and cooperating objects. It has several advantages over procedural programming:

- OOP is faster and easier to execute
- OOP provides a clear structure for the programs
- OOP makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time.

Classes and objects are the two main aspects of object-oriented programming.

A class is a structure that defines the data and the methods to work on that data. It can contain any of the following variable types.

- **Local variables**: Variables defined inside methods, constructors or blocks are called local variables. The variable will be declared and initialized within the method and the variable will be destroyed when the method has completed.
- **Instance variables**: Instance variables are variables within a class but outside any method. These variables are instantiated when the class is loaded. Instance variables can be accessed from inside any method, constructor, or blocks of that class.
- **Class variables**: Class variables are variables declared within a class, outside any method, with the static keyword.

Creating an Object:
A class provides the blueprints for **objects**. So basically, an object is created from a class. In Java, the keyword **new** is used to create new objects. There are three steps when creating an object from a class:

- **Declaration**: A variable declaration with a variable name with an object type.
- **Instantiation**: The 'new' keyword is used to create the object.
- **Initialization**: The 'new' keyword is followed by a call to a constructor. This call initializes the new object.

Example,

```
5  public class GradeBook
6  {
7     private String courseName; // course name for this GradeBook
8
9     // method to set the course name
10    public void setCourseName( String name )
11    {
12       courseName = name; // store the course name
13    } // end method setCourseName
14
15    // method to retrieve the course name
16    public String getCourseName()
17    {
18       return courseName;
19    } // end method getCourseName
20
21    // display a welcome message to the GradeBook user
22    public void displayMessage()
23    {
24       // calls getCourseName to get the name of
25       // the course this GradeBook represents
26       System.out.printf( "Welcome to the grade book for\n%s!\n",
27          getCourseName() );
28    } // end method displayMessage
29 } // end class GradeBook
```

```java
// Fig. 3.8: GradeBookTest.java
// Creating and manipulating a GradeBook object.
import java.util.Scanner; // program uses Scanner

public class GradeBookTest
{
   // main method begins program execution
   public static void main( String[] args )
   {
      // create Scanner to obtain input from command window
      Scanner input = new Scanner( System.in );

      // create a GradeBook object and assign it to myGradeBook
      GradeBook myGradeBook = new GradeBook();

      // display initial value of courseName
      System.out.printf( "Initial course name is: %s\n\n",
         myGradeBook.getCourseName() );

      // prompt for and read course name
      System.out.println( "Please enter the course name:" );
      String theName = input.nextLine(); // read a line of text
      myGradeBook.setCourseName( theName ); // set the course name
      System.out.println(); // outputs a blank line

      // display welcome message after specifying course name
      myGradeBook.displayMessage();
   } // end main
} // end class GradeBookTest
```

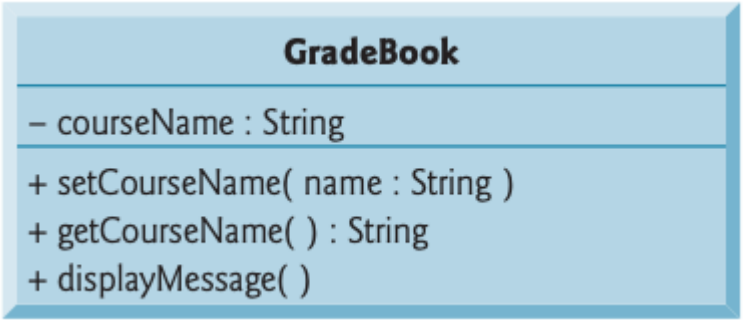
```
Initial course name is: null

Please enter the course name:
CS101 Introduction to Java Programming

Welcome to the grade book for
CS101 Introduction to Java Programming!
```

UML Class Diagram

In the UML, each class is modelled in a class diagram as a rectangle with three compartments. The top compartment contains the name of the class centred in boldface type. The middle compartment contains the class's attributes, which correspond to instance variables. The bottom compartment contains the class's operations, which correspond to methods in Java.



Constructors

It is like a method but is used only at the time an object is created to initialize the object's data.

- Constructor name == the class name
- No return type – never returns anything
- All classes need at least one constructor. If you don't write one, Java compiler will not create a default constructor for that class

```
Classname()
{

}
```

Static

• Applies to variables and methods
• Means
  – Is defined for the class declaration,
  – Is **not** unique for each instance

Note

- Each class declaration that begins with the access modifier public.
- classes, which specify the types of objects, are reference types.
- Every class declaration contains keyword class followed immediately by the class's name.
- A method declaration that begins with keyword public indicates that the method can be called by other classes declared outside the class declaration.
- By convention, method names begin with a lowercase first letter and all subsequent words in the name begin with a capital first letter.
- Empty parentheses following a method name indicate that the method does not require any parameters to perform its task.
- Every method's body is delimited by left and right braces ({ and }).
- The method's body contains statements that perform the method's task. After the statements execute, the method has completed its task.
- When you attempt to execute a class, Java looks for the class's main method to begin execution.
- Typically, you cannot call a method of another class until you create an object of that class.

Homework

Q> Using programming show the effect of the keyword ***this***.
Q> Using programming show the effect of the method overloading.
Q> Convert an object into an array of object.